

Intelligent systems for industry, supply chain and environment

LESSON 13

LLM, Multimodal models, Agents for ML
(Part A)



Outline

- Introduction to agents for ML
 - LLM
 - Multimodal models
 - Agents
 - Examples
-
- Some slides in the following are from/adapted from CME 295: Transformers & Large Language Models, Afshine Amidi and Shervine Amidi, Stanford University

Introduction to LLM for IS4

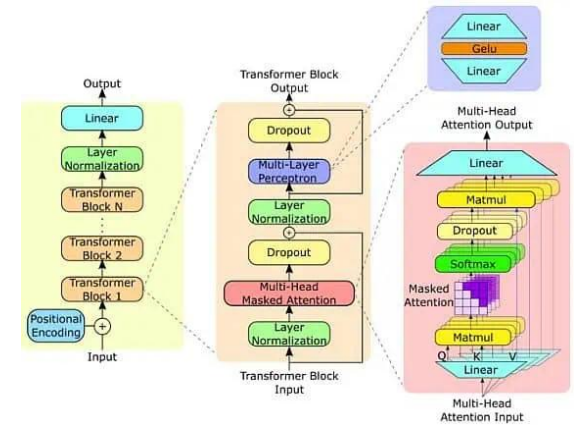


What this lecture is not

This is not a full course on:

- LLM training
- multimodal models
- detailed Transformer internals
- benchmark chasing

This lecture only covers what is needed to work with agents consciously as an ML professional.



ML vs LLM in IS4

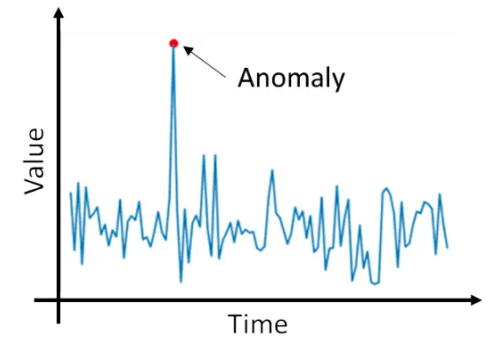
Many IS4 applications are still **classical ML**:

- forecasting
- anomaly detection
- predictive maintenance
- quality estimation
- optimization support

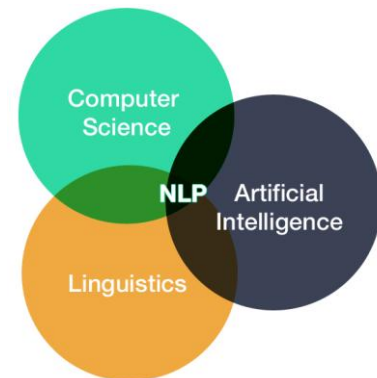
Engineering decision:

- **use classical ML** for numerical forecasting, anomaly scores, and predictive models
- **use LLMs or agents when documents, logs, code, or multi-step interaction enter the workflow**

LLMs become useful when language, documents, code, and interactive reasoning enter the loop.



NLP



- **DEFINITION**

Natural Language Processing (NLP) is a field of artificial intelligence and computer science that studies methods for enabling computers to process, analyze, understand, and generate human language.

- Natural Language Processing (NLP) enables intelligent systems to extract information, classify content, answer questions, summarize documents, and generate text from human language data.

The IS4 workflow remains the backbone

The recurring workflow is still:

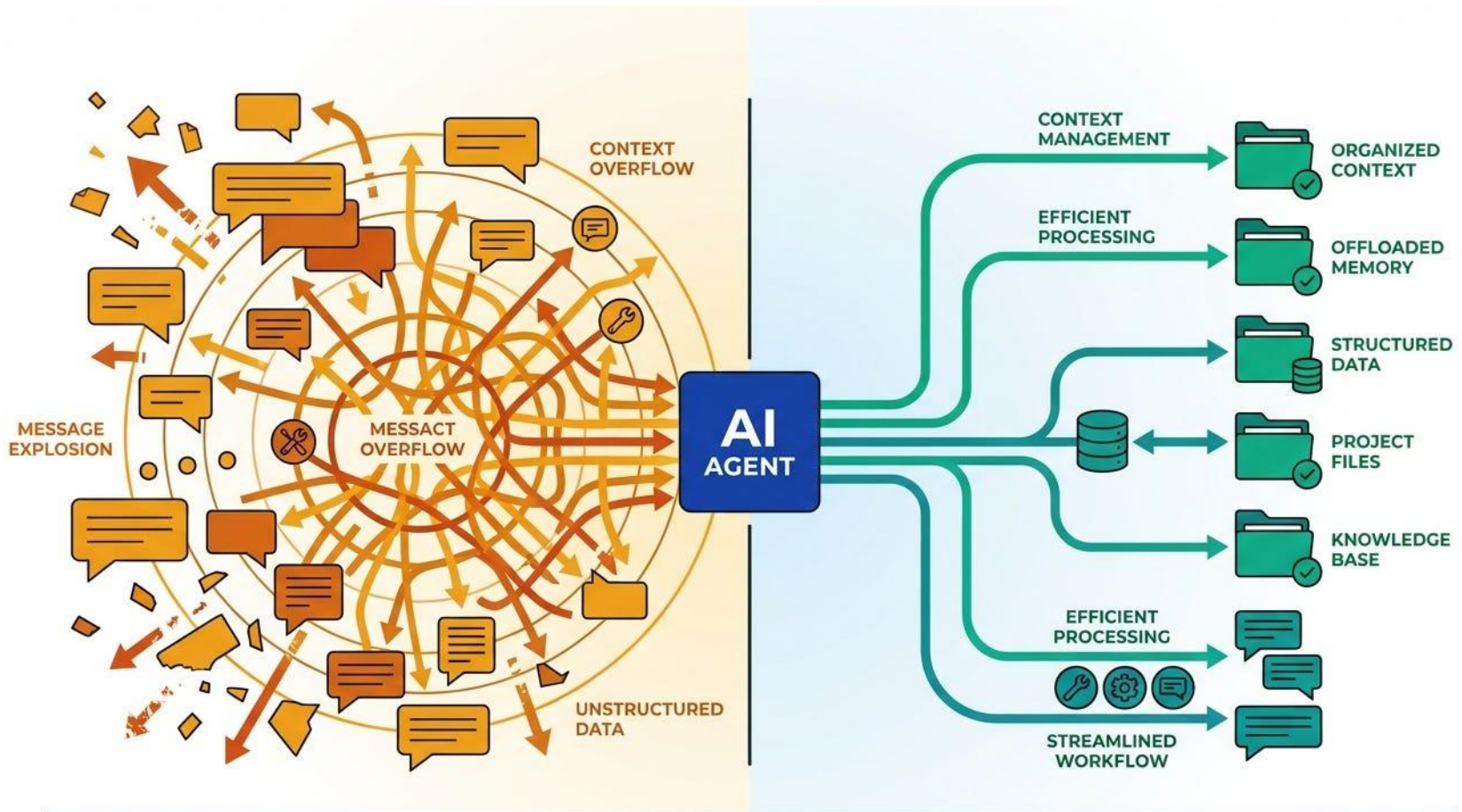
problem -> data -> representation -> model -> evaluation
-> deployment -> monitoring

LLM and agents are introduced to improve this workflow, not to replace it.

They can help with:

- project context reconstruction
- data preparation and missing data analysis
- experiment design and reporting
- monitoring, audit, and documentation

Using agents to improve the design context



CHAOS: CONTEXT EXPLOSION

CLARITY: CONTEXT MANAGEMENT

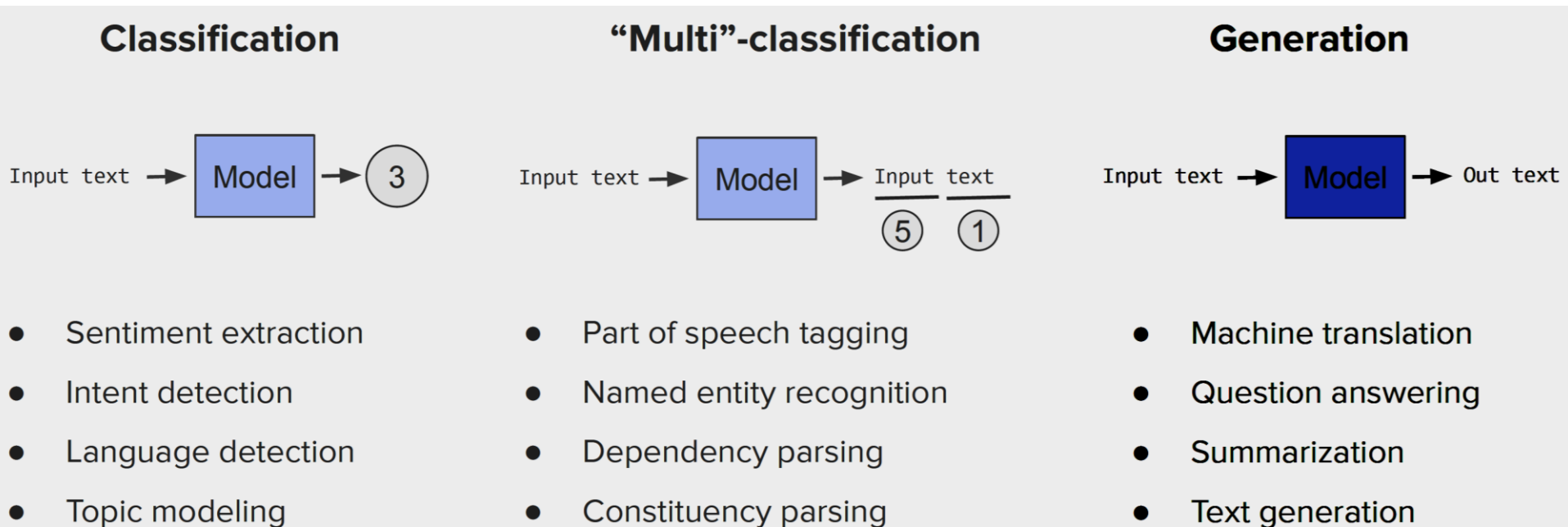
From the basics to the GPTs



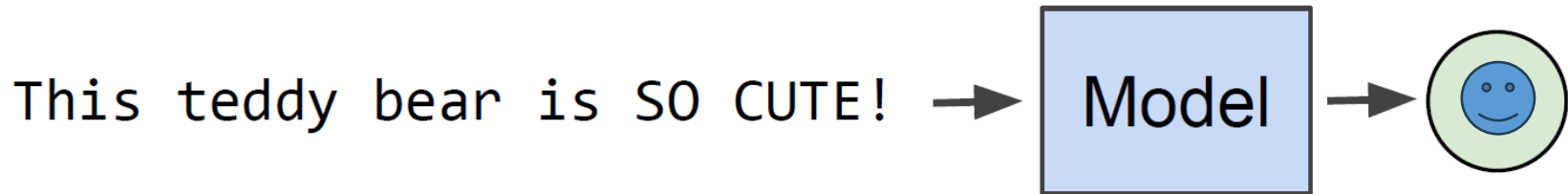
ML vs Generation (LLM)

(input is text)

This figure separates three families of language tasks. Classification produces one label for the whole input, token-level labeling produces labels for parts of the input, and generation produces new text. Modern LLMs are mainly used as generative models, but many industrial applications still need classification and token-level extraction.

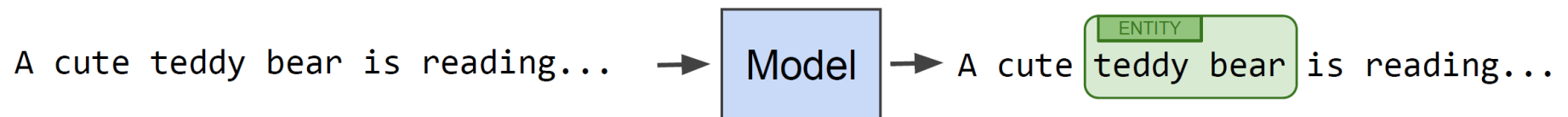


NLP task: Sentiment Extraction



- Accuracy → fraction of correct predictions over all observations
- Precision → fraction of predicted positives that are true positives
- Recall → fraction of true positives that are correctly retrieved
- F1 score → harmonic mean of precision and recall

NLP task: Named Entity Recognition

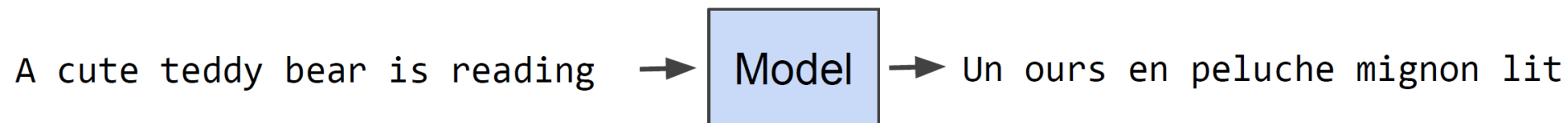


Named Entity Recognition (NER) identifies and classifies entities mentioned in text, such as names, organizations, locations, dates, codes, and quantities.

- Accuracy
- Precision
- Recall
- F1 score

NER is evaluated at token level or entity-span level, usually per entity type.

NLP task: Machine Translation



An **n-gram** is a contiguous sequence of **n words, characters, or tokens** extracted from a text)

Evaluation metrics

- BLEU → n-gram overlap with reference translations
- ROUGE → overlap with reference summaries or generated texts
- Perplexity → how unexpected a text sequence is for the model

Text:
the machine stopped suddenly

1-gram:
machine

2-gram:
machine stopped

3-gram:
the machine stopped

Tokens as the basic unit of LLM processing

- An LLM does not process text as words or sentences. It processes text as tokens

A token can be:

- a whole word
- part of a word
- a punctuation mark
- a number fragment
- a technical identifier fragment

Definition of tokens

- Definition – A token is an indivisible unit of text, such as a word, subword or character, and is part of a predefined vocabulary.
- Special tokens:
 - [UNK]
represents unknown or unseen text pieces
 - [PAD]
fills empty positions so that sequences can have the same length

Different ways to create the tokens

A cute teddy bear is reading.

A	cute	teddy bear	is	reading	.
---	------	------------	----	---------	---

A	cute	teddy	bear	is	reading	.
---	------	-------	------	----	---------	---

A	cute	ted	##dy	bear	is	read	##ing	.
---	------	-----	------	------	----	------	-------	---

A	_	c	u	t	e	_	t	e	d	d	y	_	b	e	a	r	_	i	s	_	r	e	a	d	i	n	g	.
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Tokenizer

A tokenizer T divides text into tokens at an arbitrary level of granularity.

this teddy bear is reaaaaally cute

→ $[T]$ →

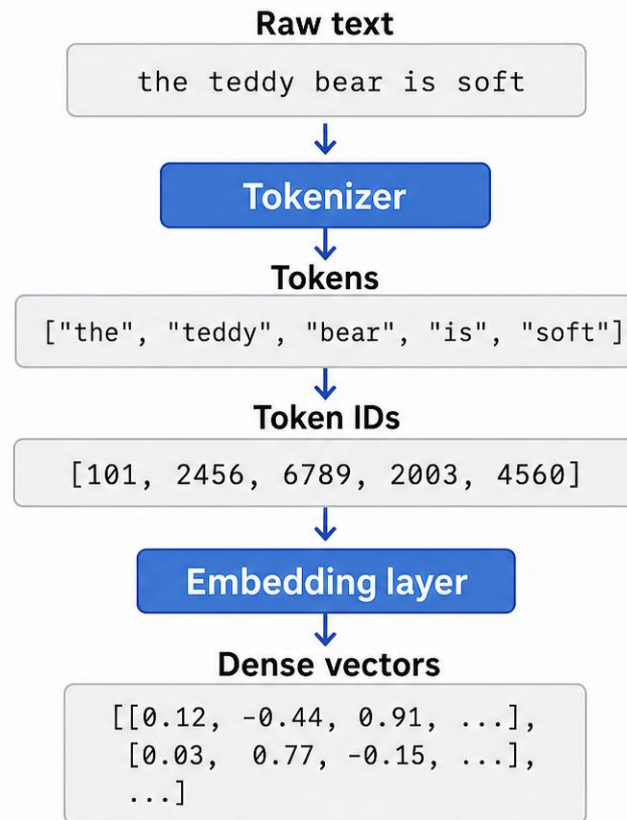
this		teddy		bear		is		[UNK]	
cute		[PAD]		...		[PAD]	XXX		

Tokenization summary

Type	Pros	Cons	Illustration
Word	Easy to interpret; short sequence	Large vocabulary size; word variations not handled	teddy ; bear
Subword	Word roots leveraged; intuitive embeddings	Increased sequence length; tokenization more complex	ted ; ##y ; bear
Character	No out-of-vocabulary concerns; small vocabulary size	Much longer sequence length; patterns harder to interpret because too low-level	t ; e ; d ; d ; y ; b ; e ; a ; r

From text → Token ID → Embeddings

From raw text to embeddings

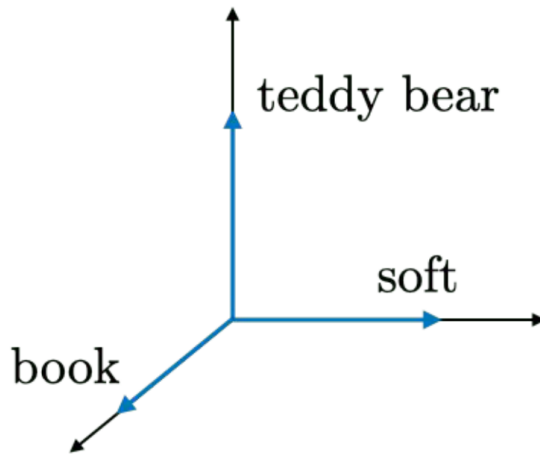


Tokenizer maps text to discrete tokens and IDs. The embedding layer maps IDs to dense numerical vectors.

Why using tokens

trivial encoding = one hot encoding

Naive (one-hot) encoding



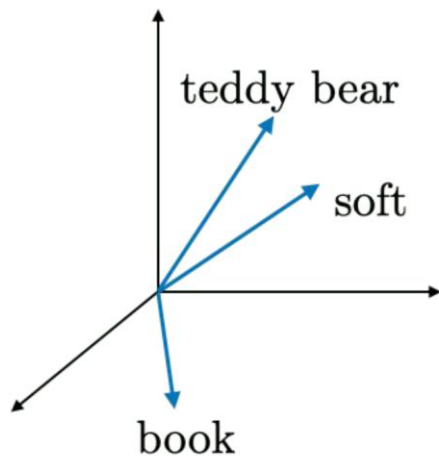
$$\text{soft} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \begin{aligned} \langle \text{teddy bear}, \text{book} \rangle &= 0 \\ \langle \text{teddy bear}, \text{soft} \rangle &= 0 \end{aligned}$$

- One-hot encoding represents each token as an independent vector.
- Different tokens are orthogonal, so semantic similarity is not captured.
 - $\text{Similarity}(\text{teddy bear}, \text{soft}) = 0$
 - $\text{Similarity}(\text{teddy bear}, \text{book}) = 0$If Similarity is the scalar product:
 - $\langle \text{teddy bear}, \text{book} \rangle = 0$
 - $\langle \text{teddy bear}, \text{soft} \rangle = 0$ (teddy is not soft!!)
- This is why(good!) embeddings are needed: they place related tokens closer in vector space.

Why using tokens

Learned/trained encoding

Learned embedding



$$\text{soft} = \begin{pmatrix} 0.95 \\ 0.32 \\ 0.01 \end{pmatrix} \quad \begin{array}{l} \langle \text{teddy bear}, \text{book} \rangle \sim 0 \\ \langle \text{teddy bear}, \text{soft} \rangle \sim 1 \end{array}$$

- Learned embeddings represent tokens as dense vectors.
- Unlike one-hot vectors, learned embeddings can encode semantic similarity.
- In this representation:
 - $\langle \text{teddy bear}, \text{soft} \rangle \approx 1$
(teddy is soft!!)
 - $\langle \text{teddy bear}, \text{book} \rangle \approx 0$
(teddy is not a book!!)
- This means that "teddy bear" is closer to "soft" than to "book" in the learned vector space.

Embeddings

Definition – An embedding is a numerical representation of an element (e.g. token, sentence) and is characterized by a vector $x \in \mathbb{R}^n$.

Token String	Token ID	Embedded Vector
"The" →	101	→ [0.5, 0.8, ..., 0.65]
"cat" →	2598	→ [0.1, 0.3, ..., -0.82]
"sat" →	4450	→ [0.41, -0.53, ..., 0.8]
"." →	119	→ [0.17, 0.75, ..., 0.2]

Similarity (cosine)

The *cosine similarity* between two tokens t_1, t_2 is quantified by

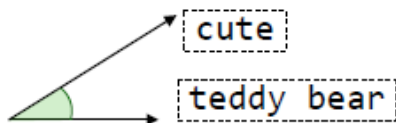
$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{||t_1|| \ ||t_2||} = \cos(\theta) \in [-1, 1]$$

The **angle θ** characterizes the similarity between the two tokens

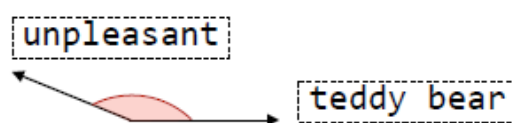
Similarity (cosine)

- The angle θ characterizes the directional similarity between two tokens or feature vectors.
- Unlike Euclidean distance, cosine similarity ignores vector magnitude and compares the orientation of vectors in the embedding space.

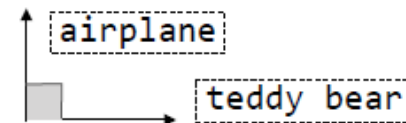
Similar



Dissimilar



Independent



The token economy

Even the most advanced LLMs still process text through tokens. Tokens are not only a technical representation. They are also used to estimate:

- workload
- usage
- latency
- billing

Tokens are both a computational unit and an economic unit.

CURRENT TOKEN PRICING (December 2025)

Per 1 Million Tokens

GPT-5 Nano	Gemini Flash-Lite	GPT-5 Mini	DeepSeek R1	GPT-5
 BUDGET	 VALUE	 MID-TIER	 REASON	 PREMIUM
Input \$0.05	Input \$0.075	Input \$0.25	Input \$0.55	Input \$1.25
Output \$0.40	Output \$0.30	Output \$2.00	Output \$2.19	Output \$10.00



TOKENS ARE THE NEW CURRENCY

Like the HTTP request enabled the web economy, **tokens are the atomic unit of value** in the AI economy.
Pricing in \$/million tokens. Output costs 4-10× input because generation is sequential, not parallel.

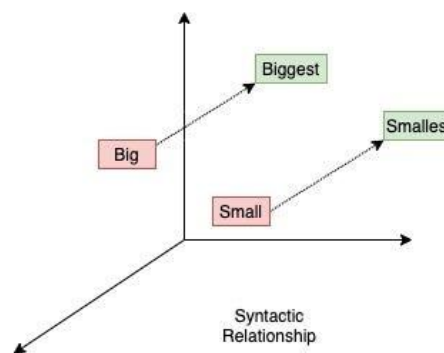
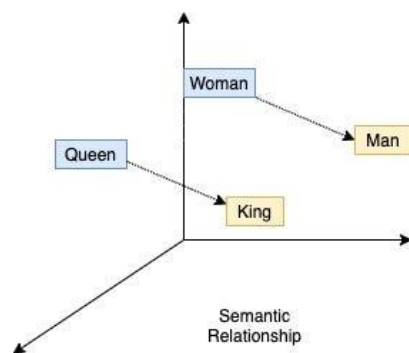
Note: tokens are simultaneously a unit of computation and a unit of cost; this dual role explains why design decisions about prompts and context have direct economic consequences.

word2Vec

(there are different version of it)

- Word2Vec maps words from a vocabulary into dense numerical vectors learned from word co-occurrence in large text corpora.
- Words that appear in similar contexts tend to obtain similar vectors.

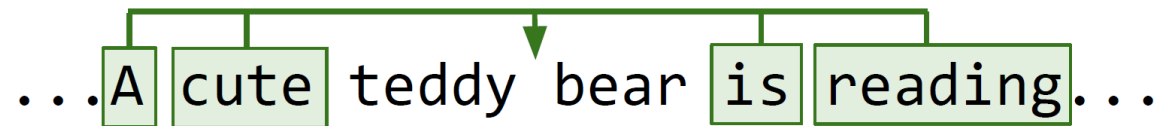
oneword $\rightarrow$ [Word2Vec] $\rightarrow$ vector



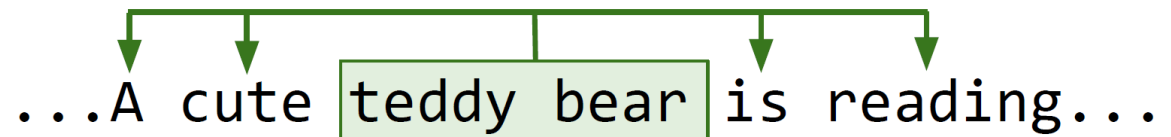
word2Vec: training

- Word2Vec learns word vectors by looking at how words appear near each other in large collections of text.

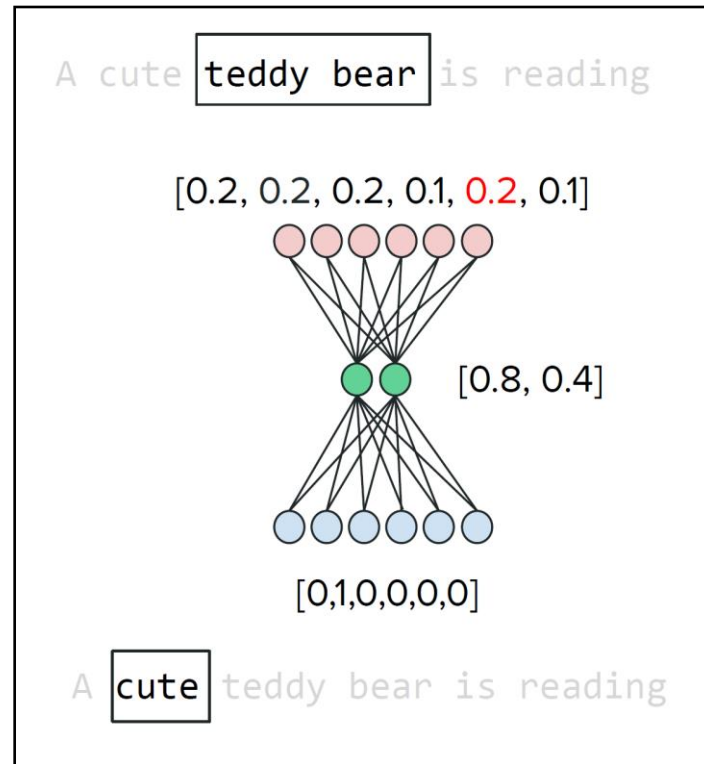
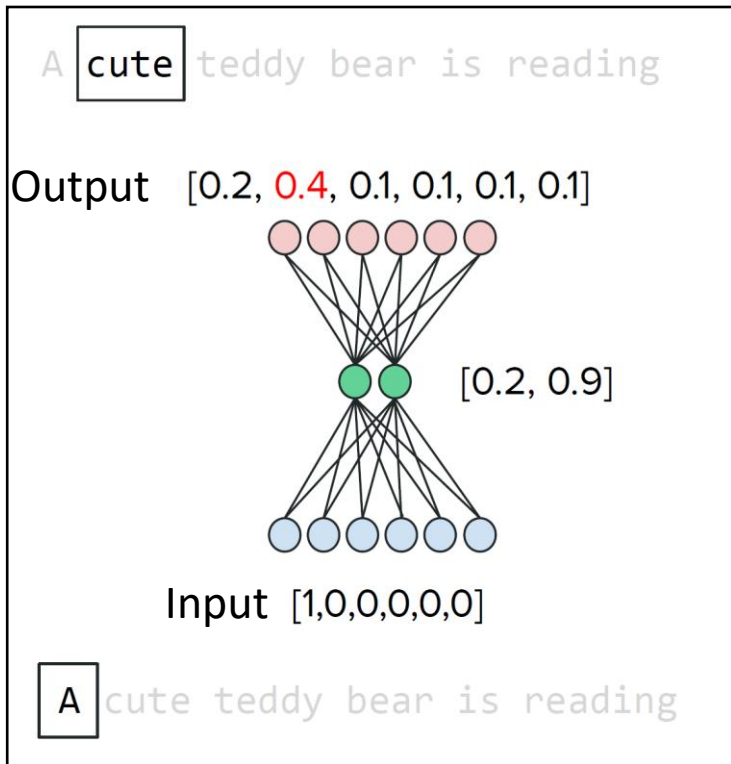
CBOW (continuous bag of words)



Skip-gram



word2Vec: architecture and training



The network is trained to predict words from their context.

The input word is first represented as a sparse vector, such as a one-hot vector.

Inside the network, this sparse representation is compressed into a smaller dense representation.

This internal representation is the latent space.

After training, the coordinates in this latent space become the word embedding vector. "A" $\rightarrow$ [0.2, 0.9], "cute" $\rightarrow$ [0.8, 0.4], ..

Recurrent Neural Networks (RNNs)

the precursors of LLM

Sequence is important!

- words in a sentence
- sensor readings over time
- machine events in a log
- user actions in a process

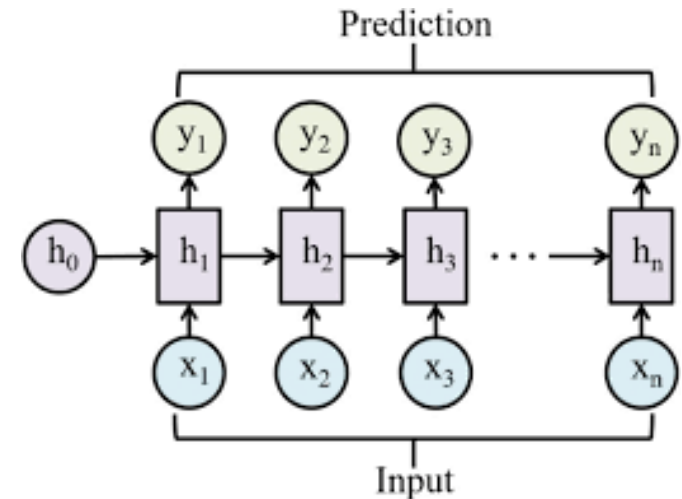
Example:

- "the machine stopped after the alarm"
- is different from:
- "the alarm stopped after the machine"

A standard feed-forward neural network does not naturally remember what came before.

Recurrent Neural Networks were introduced to process sequences step by step, while keeping an internal state that carries information from previous inputs.

Un ours en peluche mignon lit



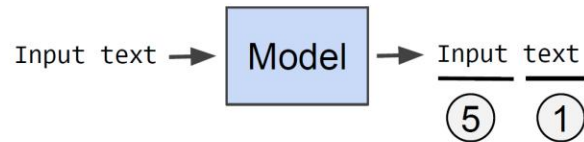
A cute teddy bear is reading

Recurrent Neural Networks (RNNs)

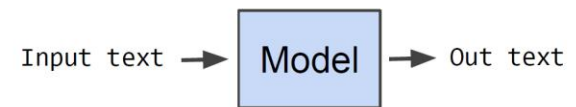
Classification



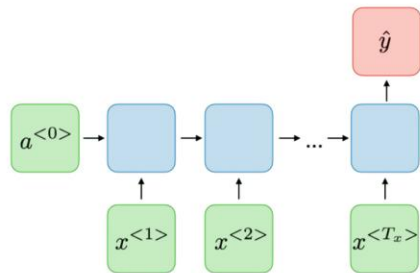
"Multi"-classification



Generation

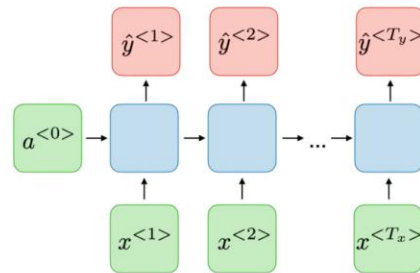


Sentiment



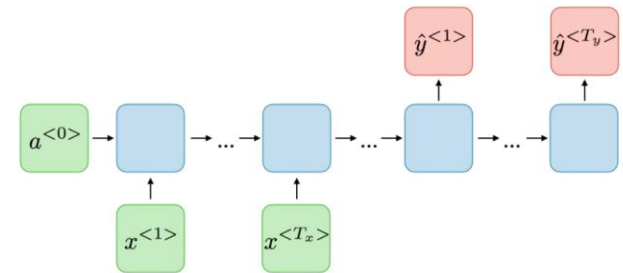
Opinion

Tags



Text

Translation



Source

Sentence:

"The movie was great"

$x^{<1>} = \text{"The"}$

$x^{<2>} = \text{"movie"}$

$x^{<3>} = \text{"was"}$

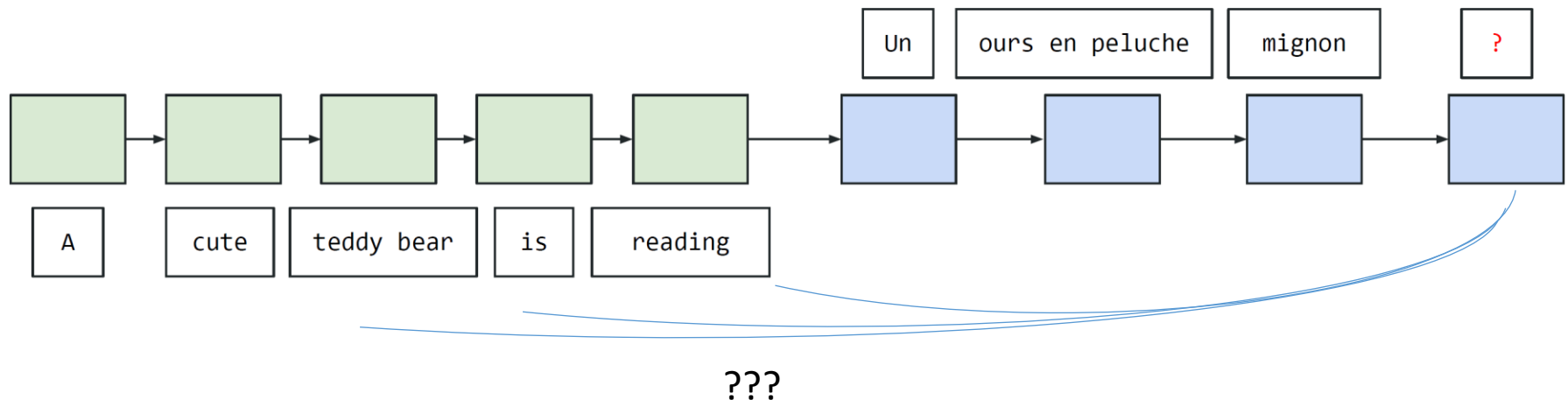
$x^{<4>} = \text{"great"}$

$T_x = 4$

The need of attention!

What should I look at in the input tokens?

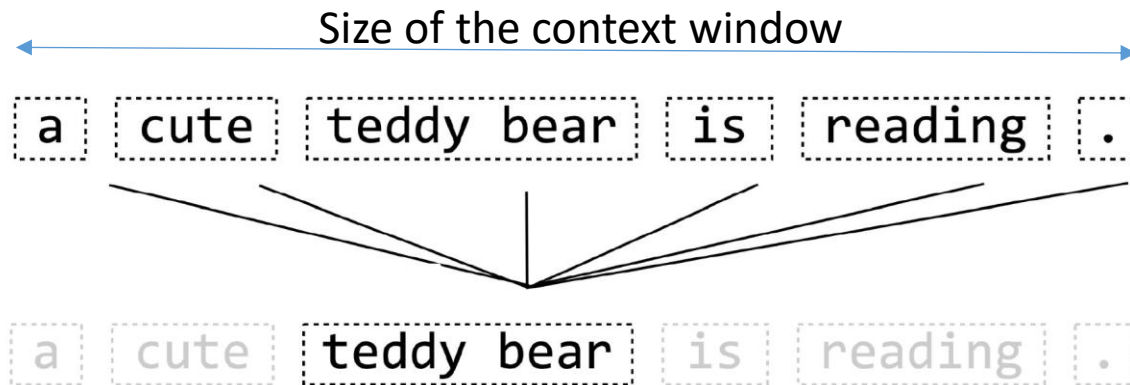
- Translation tasks had a real issue with long-term dependencies
- Classical text converters like RNN (seq2seq) may fail to "remember" what input sentence was saying



Solution1: self-attention

It creates the “context”

- Introduced in the 2017 paper "Attention is All You Need"
- State-of-the-art results on machine translation tasks



Self-attention allows each token to look at the other tokens in the same sequence.

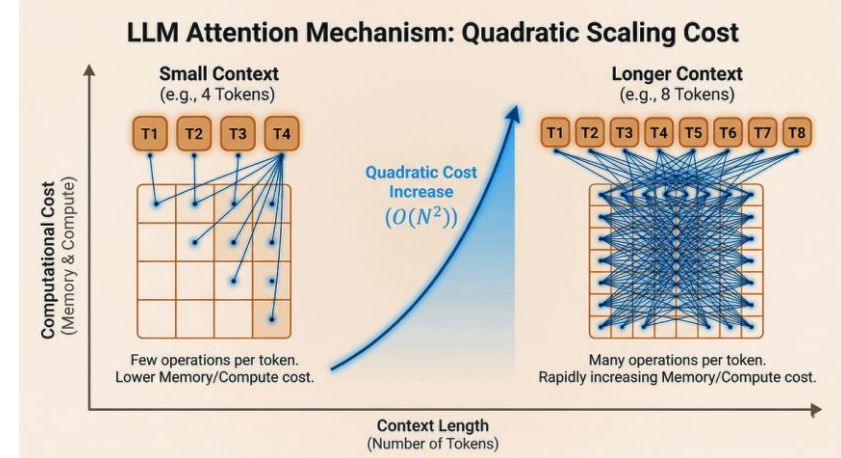
In this example, the model is building the representation of the token "teddy bear".

To do that, it can attend to surrounding tokens such as "cute", "is", and "reading".

The lines represent possible attention connections between the current token and the other tokens.

The final representation of "teddy bear" is therefore context-dependent, not isolated.

Attention is quadratic...

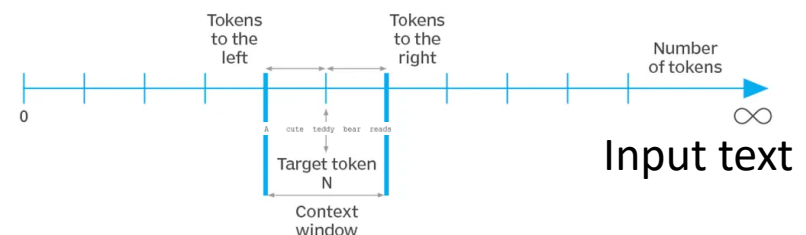


- Standard self-attention creates an $n \times n$ attention matrix for a sequence of n tokens.
- Each entry measures how much one token attends to another token in the current context window.
- This allows all tokens to interact, but the computational cost grows as $O(n^2)$.

Example with 5 tokens: A cute teddy bear reads

	A	cute	teddy	bear	reads
A	0.10	0.20	0.30	0.20	0.20
cute	0.05	0.10	0.45	0.30	0.10
teddy	0.05	0.20	0.25	0.40	0.10
bear	0.05	0.10	0.50	0.25	0.10
reads	0.10	0.15	0.35	0.30	0.10

Example of a context window



Attention matrices are generally not symmetric.

Missing a piece: positional encoding

Attention alone does not know
the order of the sequence.

Example:

the machine stopped
stopped the machine

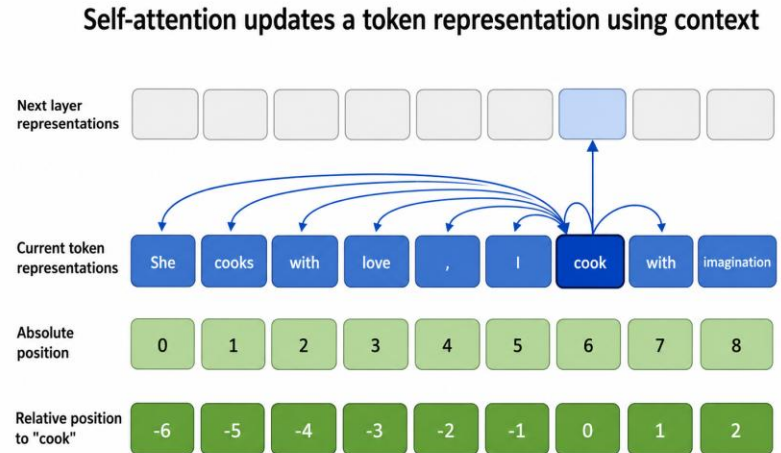
The same words appear, but the meaning changes because the
order changes.

Positional encoding adds information about token position to the
token embedding.

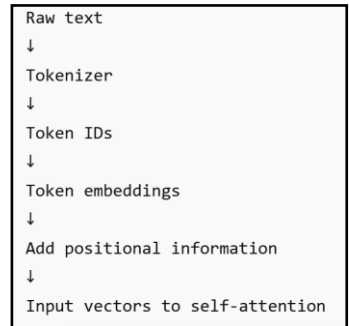
token representation = token embedding + positional encoding

This allows the Transformer to process both:

- what the token is
- where the token is in the sequence

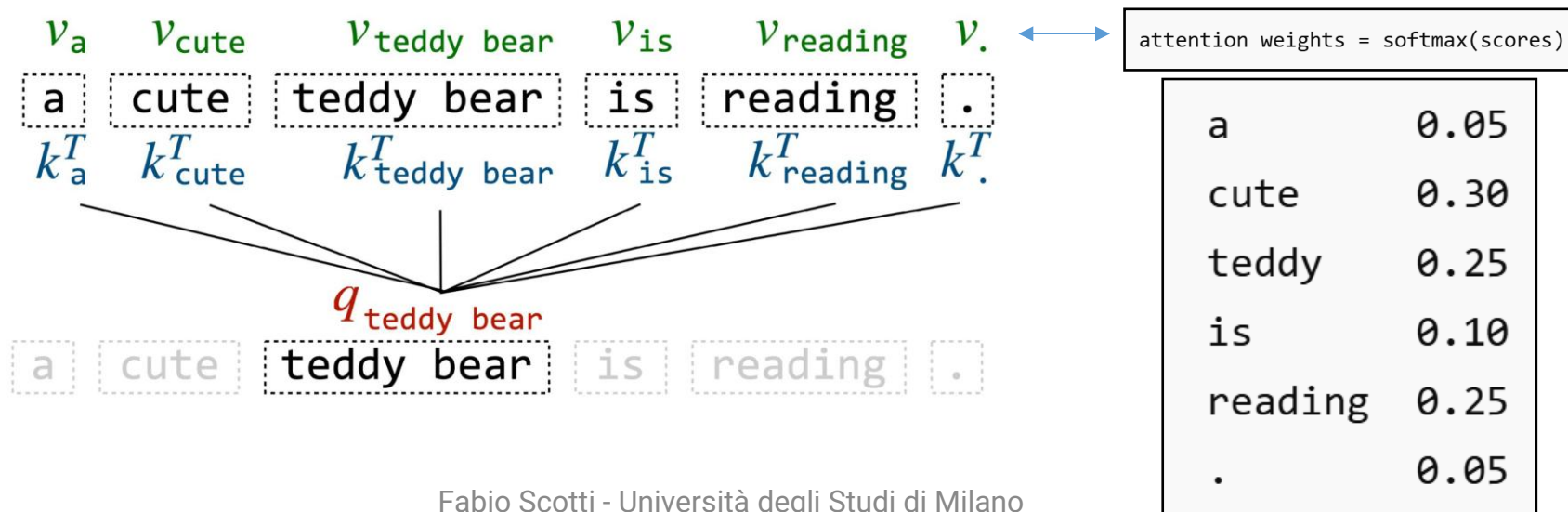


The model operates on token representations enriched with position information, not on raw words alone.



Attention: Query, Key, Value

- To compute attention, each token representation is projected into three learned vector spaces called Query, Key, and Value.
 - **Query** and **Key** determine how much tokens attend to each other, while
 - **Value** contains the information combined in the output representation.



Attention softmax

(not in the exam)

- The attention output is obtained by computing a probability distribution over the tokens through softmax, and then using this distribution to form a weighted sum of the Value vectors.

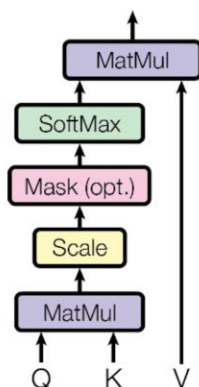
Efficient computations with matrices:

a	0.05
cute	0.30
teddy	0.25
is	0.10
reading	0.25
.	0.05

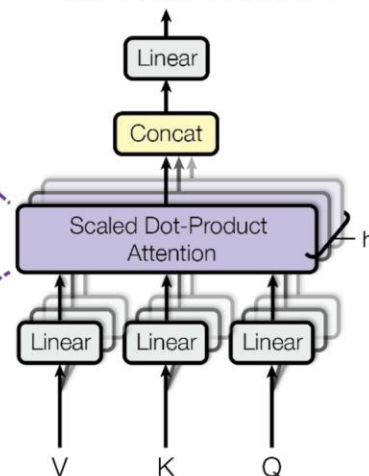
$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

dimension of
each Key vector

Scaled Dot-Product Attention

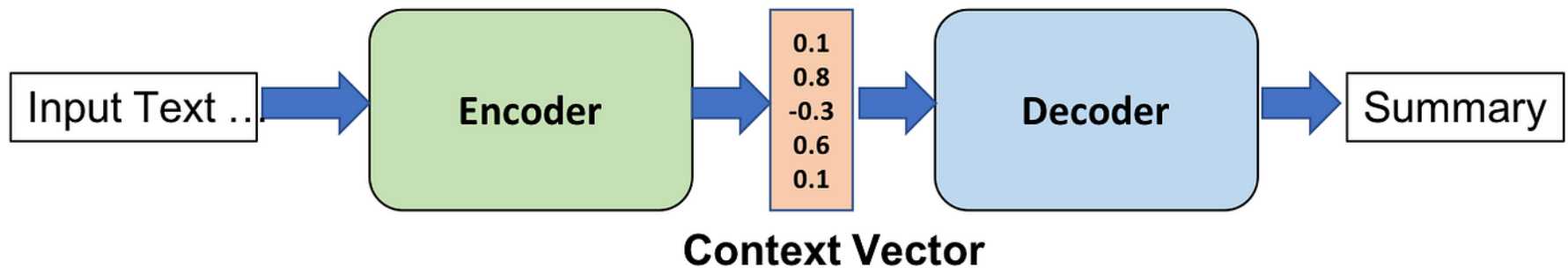


Multi-Head Attention

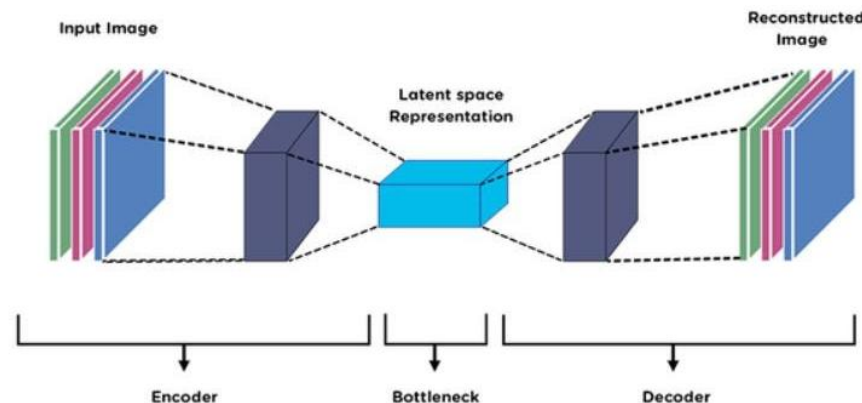


Solution2: encoder-decoders

- An encoder-decoder model transforms an input sequence into an output sequence by first encoding the input into a compact numerical representation and then decoding it into the desired output.



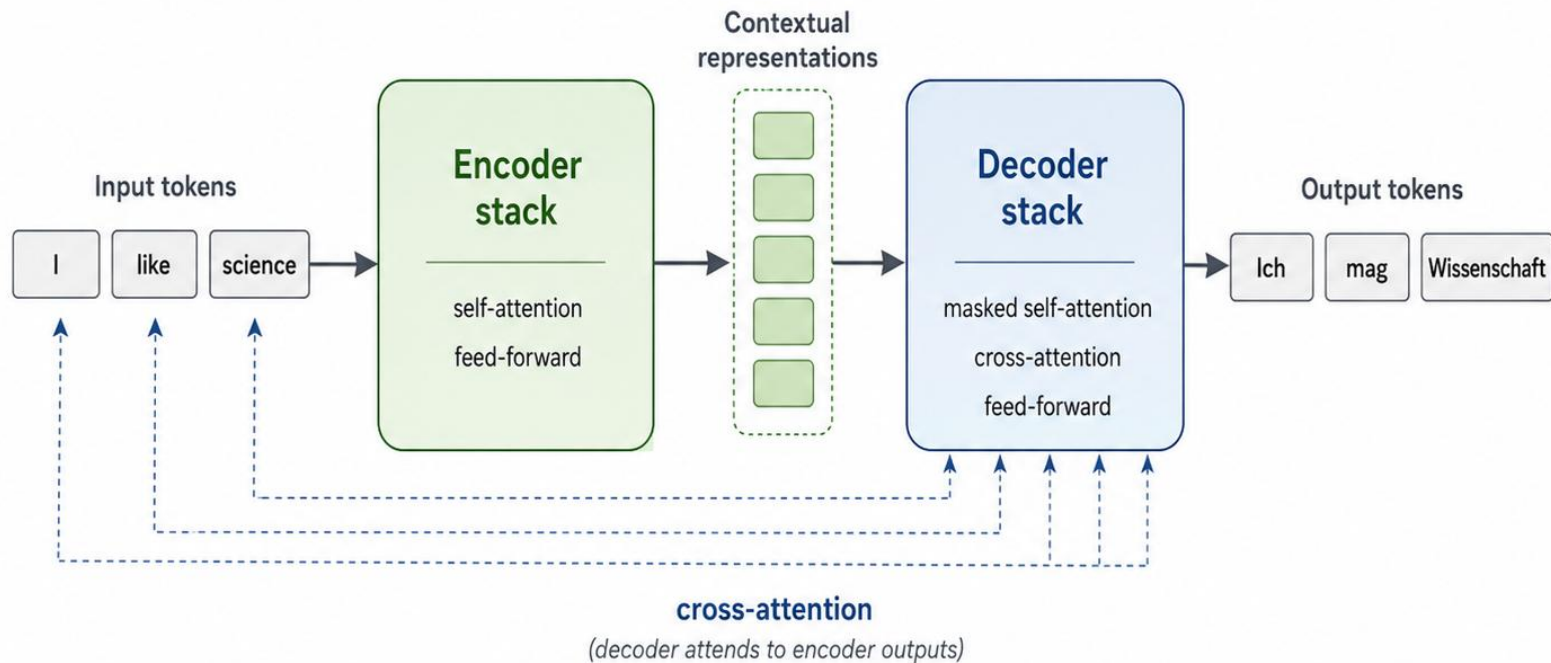
Also for images



Welcome Transformers

Transformer = attention-based encoder-decoder

Original Transformer architecture



1

Encoder reads the input sequence

2

Attention highlights relevant tokens

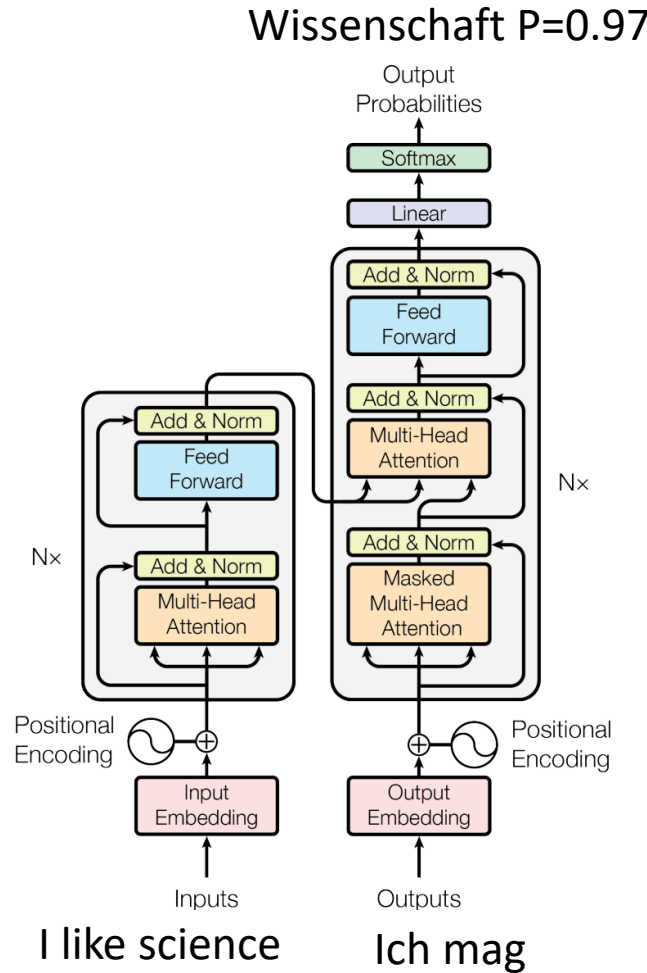
3

Decoder generates the output sequence

The original Transformer and the GPT

BERT

only
Encoder



GPT

only
Decoder

GPT = Generative Pre-trained Transformer.

It is a Transformer-based language model pre-trained on large text corpora and used to generate text one token at a time.

Modern LLM and multimodal models



From the transformers to modern LLM



- GPT introduced the idea of a Generative Pre-trained Transformer: a Transformer-based model trained on large text corpora and used to generate text token by token.
- Modern systems such as ChatGPT, Gemini, Claude, and coding assistants are built around the same general family of models:
- Large Language Models (LLMs)

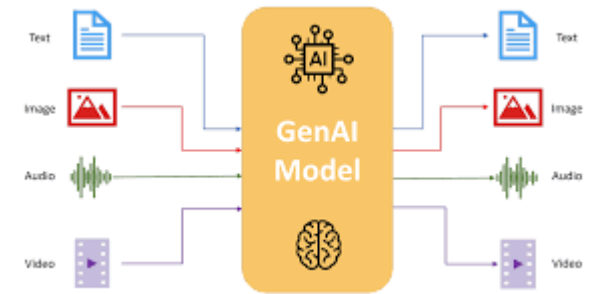
Definition:

- An LLM is a large neural language model trained on massive text data to process, generate, summarize, translate, classify, and reason over language.

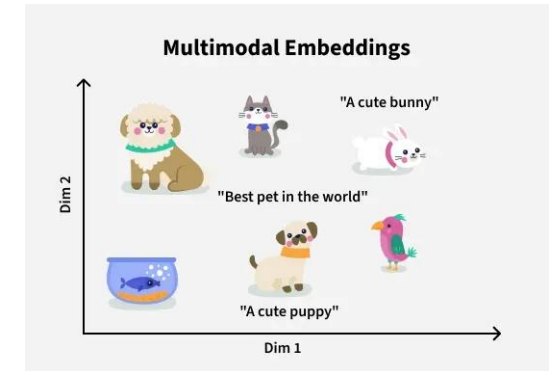
Key point:

- The basic input and output are still sequences of tokens.

From LLMs to multimodal models



- Modern multimodal models can process different input modalities:
 - text
 - images
 - audio
 - video
 - code
 - documents
 - sensor-like structured data, when properly represented
- The model does not directly "see" an image as humans do.
- The image is converted into numerical representations that can interact with text representations.
- Text becomes tokens.
- Images become visual patches or visual embeddings.



Input and output modalities

- A multimodal AI system may combine different input and output channels.
- Examples:

text → **text**

chatbot answer, summary, report

image → **text**

visual question answering, defect description

text → **image**

diagram, synthetic image, visual concept

audio → **text**

speech transcription

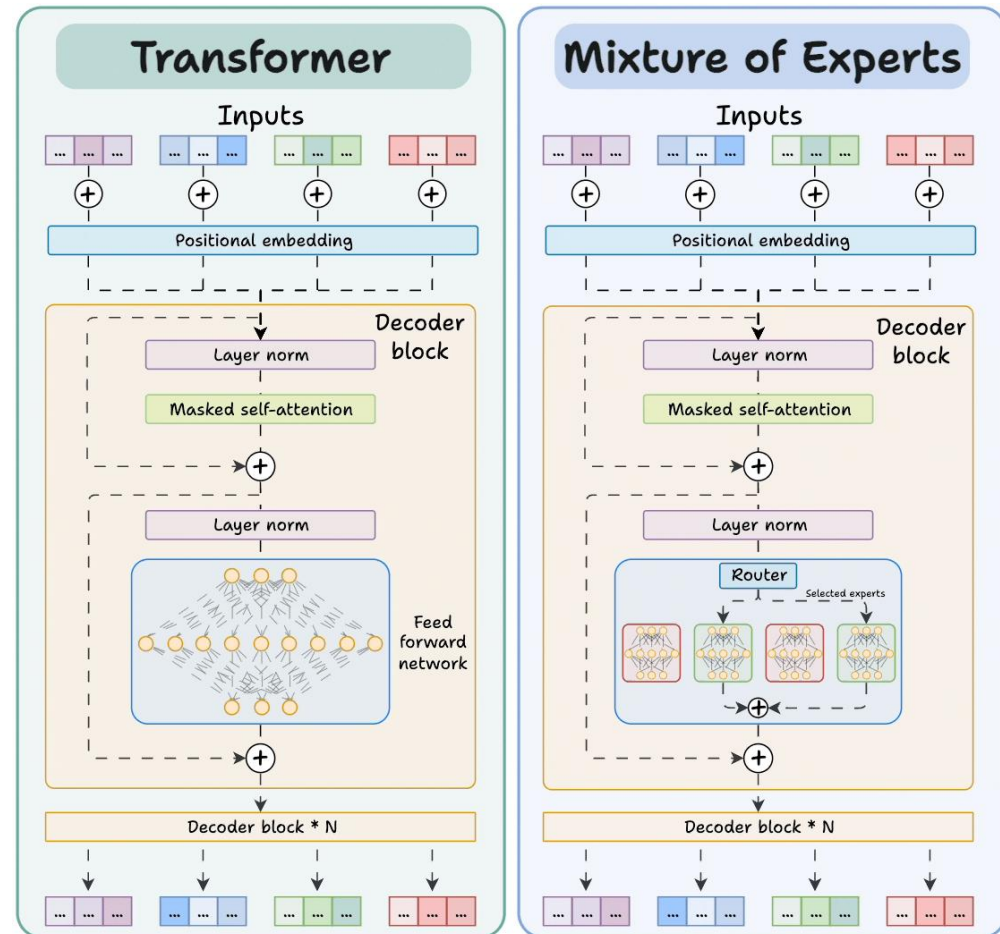
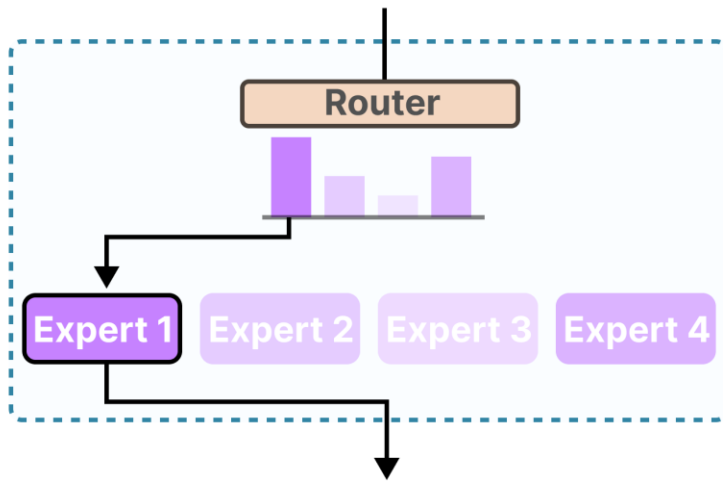
text + image → **structured data**

inspection report, JSON record, table

text + tools → **action**

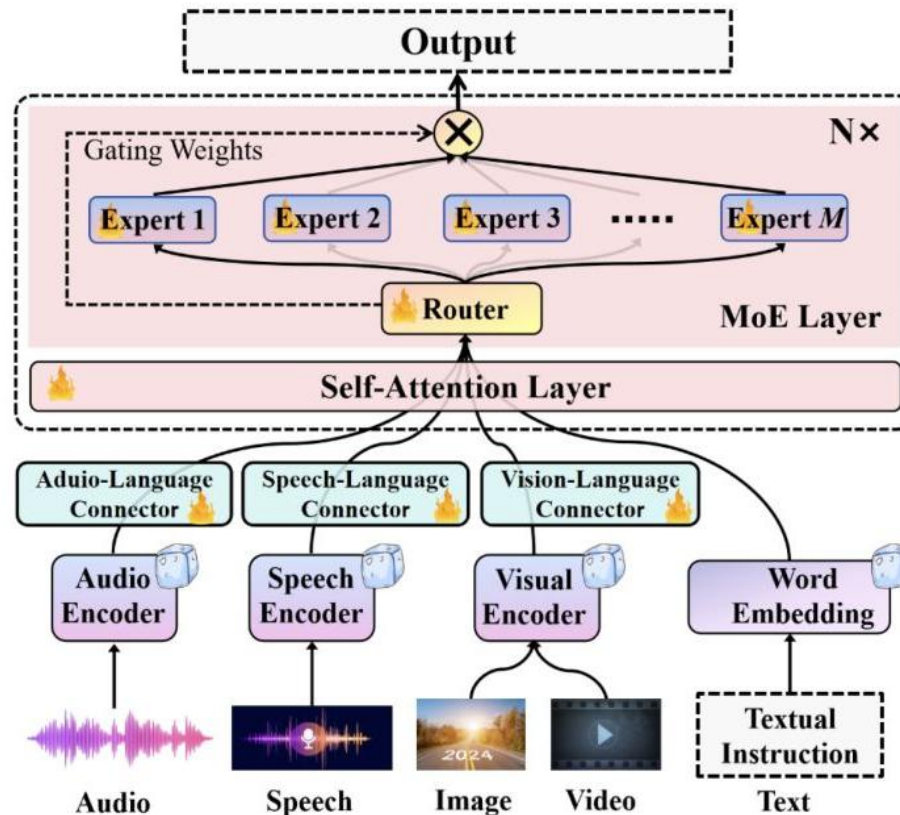
create ticket, run script, query database

Mixture of experts



Mixture of experts

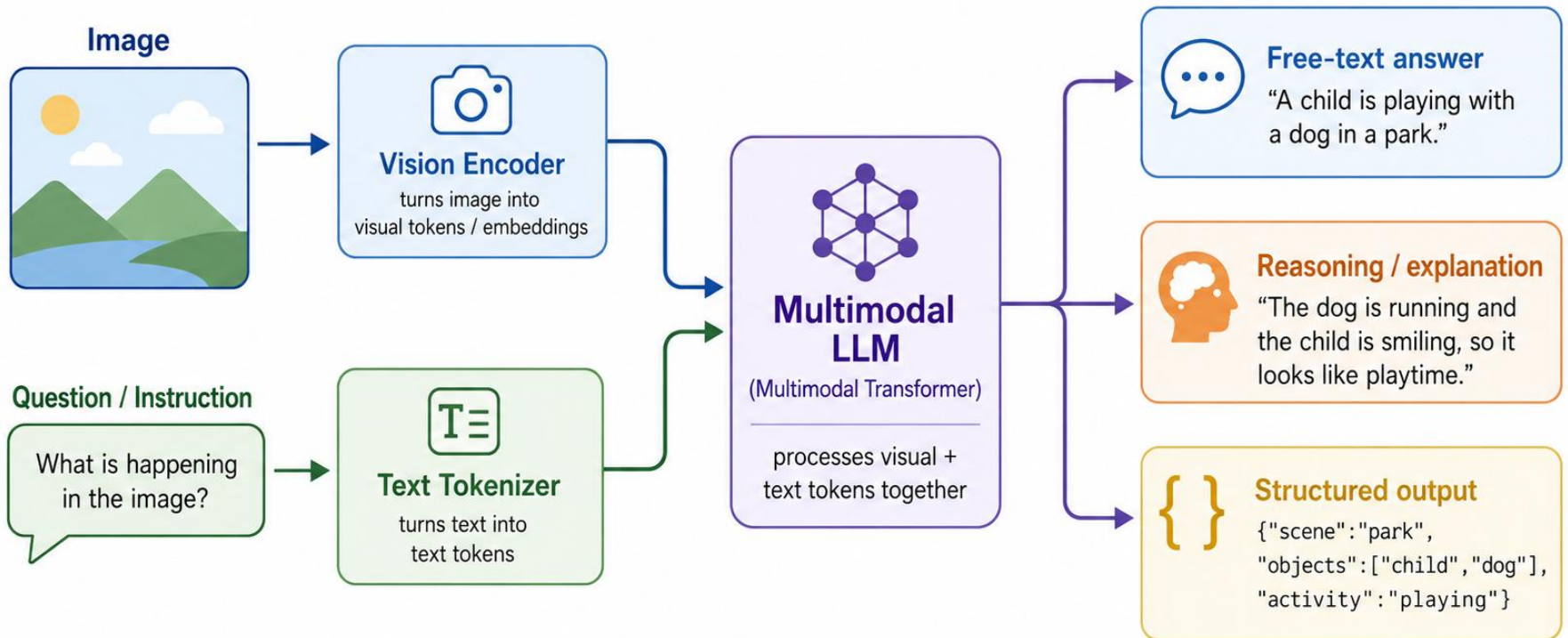
GPT-4o architecture?



Promotable models

Simple Multimodal LLM

Image + question or instruction -> free text, reasoning, structured output



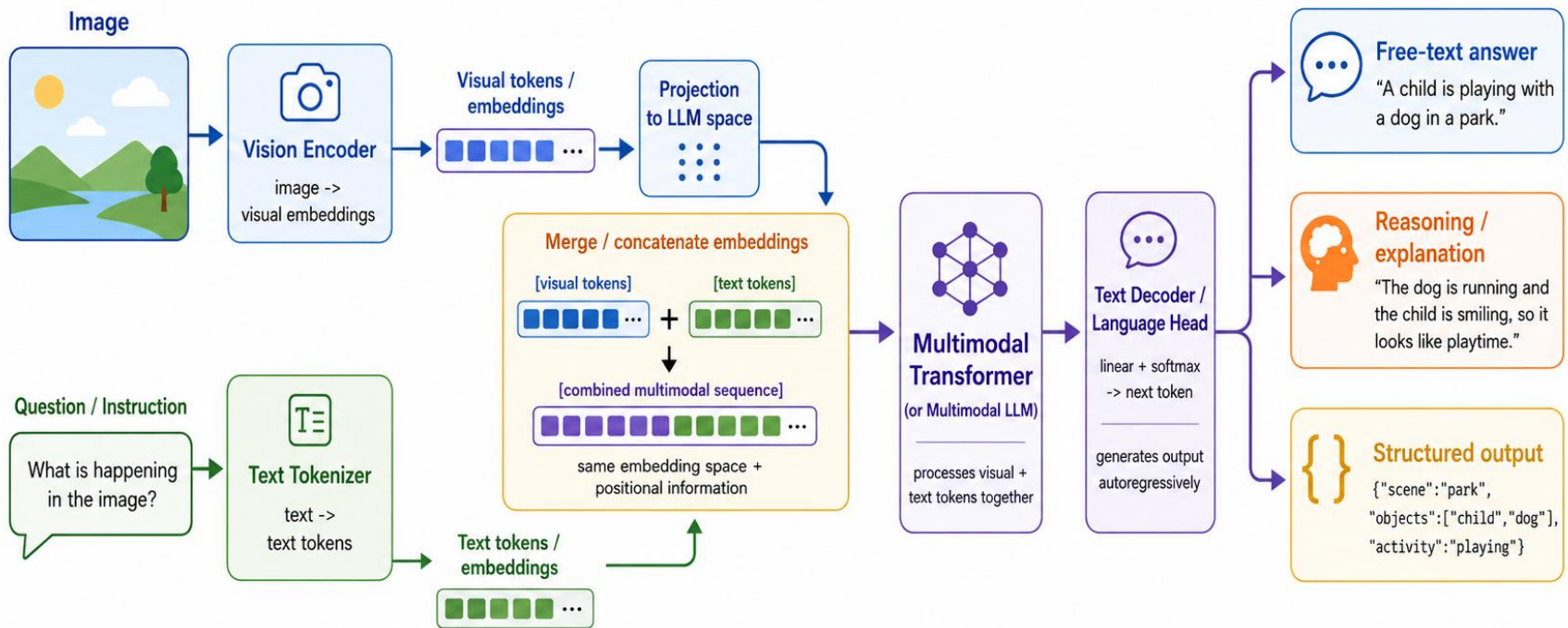
Same model, different output styles depending on the task.

Promotable models

(LLaVA models)

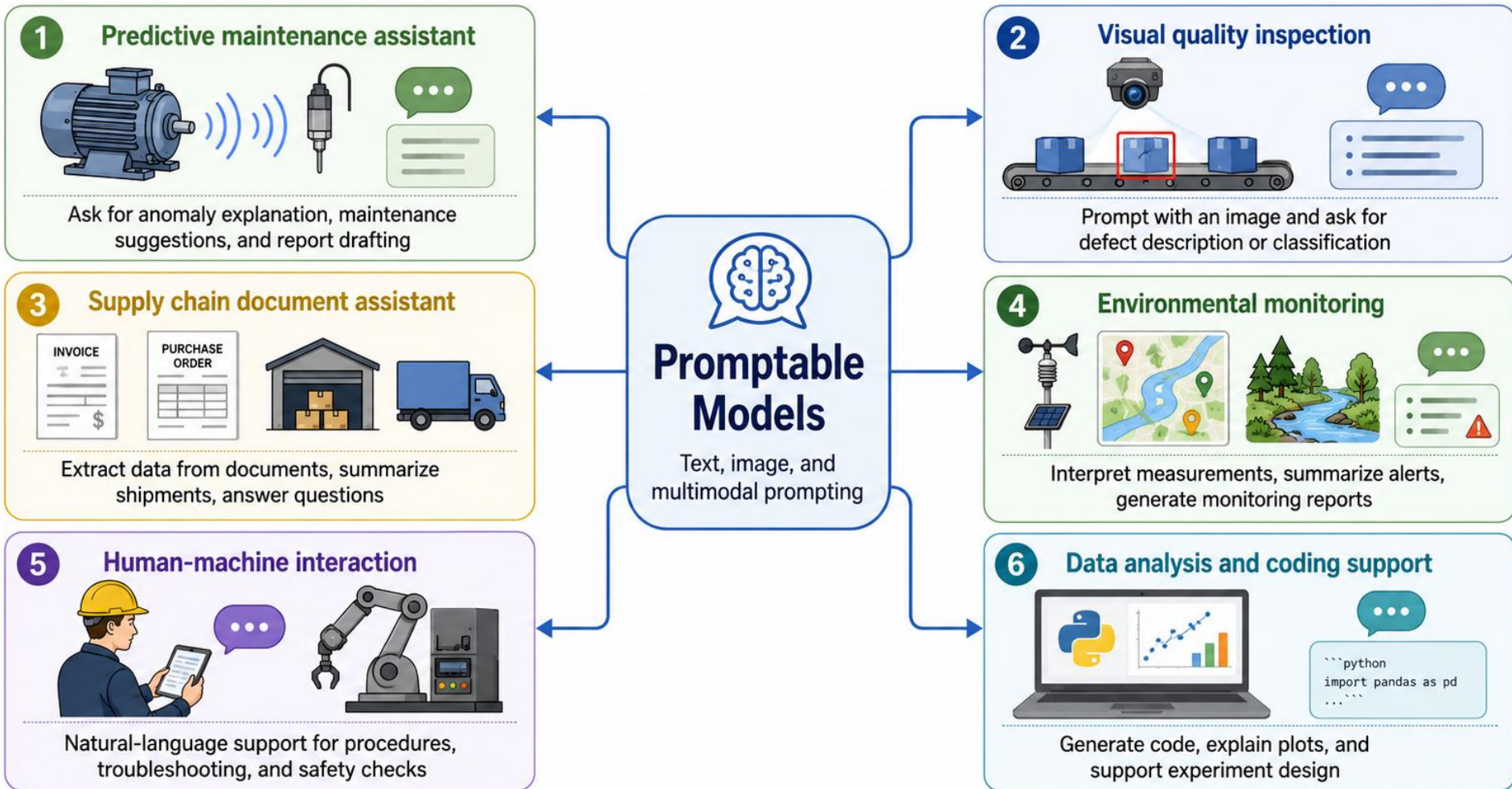
Simple Multimodal LLM

Image + question/instruction -> multimodal tokens -> decoder -> text output



Typical idea: image embeddings are projected into the language model space, merged with text tokens, and a decoder generates the output text.

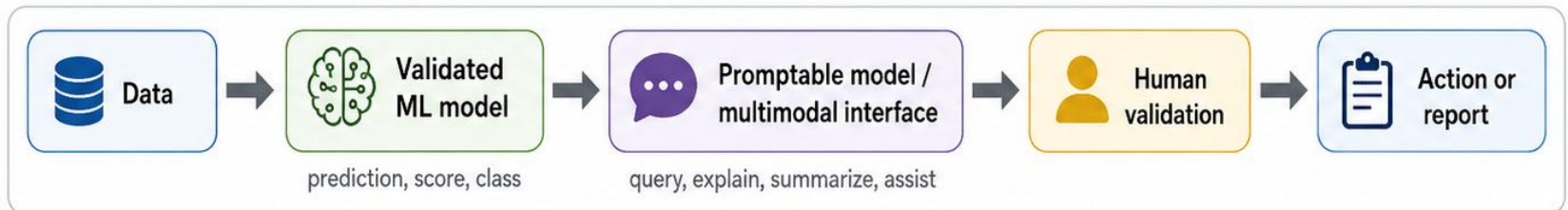
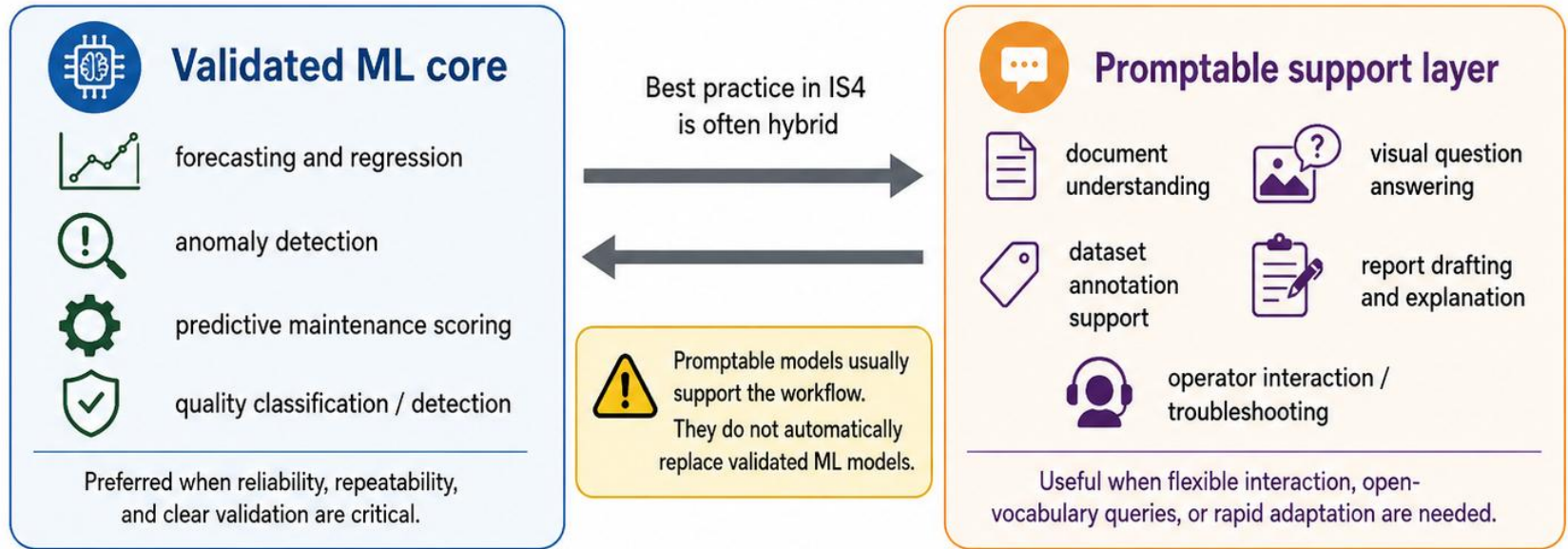
Examples of IS4 Uses of Promptable Models



In critical deployment or real-time applications, promotable models are often not suitable as the main decision engine. In these cases, it is usually **preferable to use trained and validated ML models with clearly measured performance**, stable inputs and outputs, and deterministic deployment constraints.

Promptable Models as Support Tools in IS4

IS4 = Intelligent Systems for Industry, Supply Chain and Environment



Note 1

Promptable models are strong interface and support tools, especially for documents, images, and human interaction.



Note 2

In many industrial tasks, pure ML is still preferred for higher reliability, stability, and easier performance assessment.



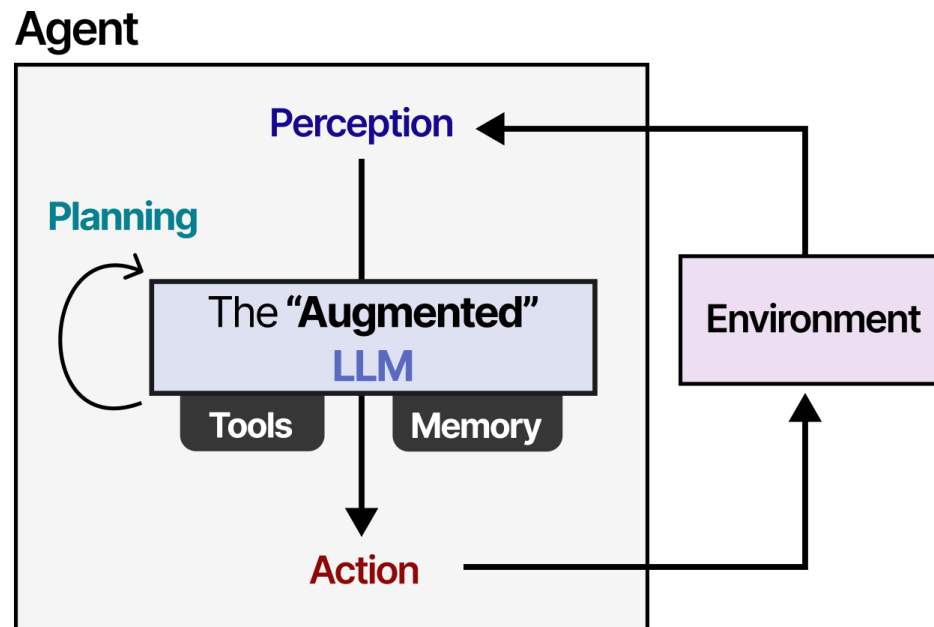
Note 3

A robust IS4 solution often combines both approaches: ML for prediction, promptable models for explanation, retrieval, and reporting.

Next lessons

What is an agent? How to use agents in IS4?

- An agent is not only a language model.
- It is an LLM placed inside a workflow loop with context, memory, tools, checks, and possibly human approval.



Main points



- **LLMs** are useful support tools, not replacements for validated ML models.
- Text is processed as **tokens**, then mapped to dense embedding vectors.
- **Embeddings** represent semantic similarity in a numerical vector space.
- **Self-attention** lets tokens interact inside the context window.
- **Positional encoding** adds information about token order.
- **Query, Key, and Value projections** compute and apply attention.
- **Transformers** combine embeddings, positional information, attention, and feed-forward layers.
- **Multimodal models (LMM)**
- We are ready to start using agents!